Student ID :	24CS047	Student Name:	Manav Lakhani			
Subject code & Name :	ITUE301: Advance Web Development Framework	Practical :	5	Academic Year	:	2026-27

Task - 1 (Practical - 5)

Aim

To connect a MongoDB database to an Express server and enforce data validation through Mongoose schema, replacing in-memory array storage with database persistence.

Code

1. models/Task.js (Mongoose Schema, Enums, and Pre-Save Hook):

```
const mongoose = require('mongoose');

/**
 * Task Schema Definition (Practical 5)
 * Fields:
 * - title: String, required
 * - description: String
 * - completed: Boolean, default false
 * - priority: String, enum ['low', 'medium', 'high'], default 'medium' (Supplementary Problem 1)
 * - createdAt: Date, default Date.now
 */
const taskSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Task title is required']
  },
  description: {
    type: String,
    default: ''
  },
  completed: {
    type: Boolean,
    default: false
  },
  priority: {
    type: String,
    enum: {
      values: ['low', 'medium', 'high'],
      message: 'Priority must be one of: low, medium, high'
    },
    default: 'medium',
    lowercase: true
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
```

```
});

// Supplementary Problem 2: Pre-save hook that automatically trims whitespace from the title field
taskSchema.pre('save', function () {
  if (this.title && typeof this.title === 'string') {
    this.title = this.title.trim();
  }
});

module.exports = mongoose.model('Task', taskSchema);
```

2. server.js (MongoDB Connection, CRUD Endpoints, and Structured Error Handling):

```
/**
 * Charotar University of Science and Technology (CHARUSAT)
 * CSPIT - Computer Science & Engineering
 * Course: Advanced Web Development Frameworks (ITUE301)
 * Practical 5: MongoDB Integration and Schema Design with Mongoose
 * Student: Manav Lakhani (24CS047)
 */

require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const Task = require('./models/Task');

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/task_manager_db';

// ========================================
// 1. Built-in JSON Body Parsing Middleware
// ========================================
app.use(express.json());

// ========================================
// 2. Global Request Logging Middleware
// Logs HTTP Method, Request URL, and Timestamp
// ========================================
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.url}`);
  next();
});

// ========================================
// 3. Content-Type Validation Middleware
// Rejects POST / PUT requests missing 'application/json' Content-Type
// ========================================
const requireJsonContent = (req, res, next) => {
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.toLowerCase().includes('application/json')) {
      return res.status(400).json({
        error: 'Bad Request',
```

```
        message: 'Requests with payload (POST/PUT) must include "Content-Type: application/json"
header.'
      });
    }
  }
  next();
};

app.use(requireJsonContent);

// ========================================
// 4. Route-Specific ID Validation Middleware
// Validates 24-character hexadecimal MongoDB ObjectId format
// ========================================
const validateTaskId = (req, res, next) => {
  const { id } = req.params;
  if (!mongoose.Types.ObjectId.isValid(id)) {
    return res.status(400).json({
      error: 'Invalid Task ID',
      message: `The provided ID '${id}' is not a valid 24-character MongoDB ObjectId.`
    });
  }
  next();
};

// ========================================
// 5. RESTful CRUD Routes for Tasks (Mongoose Model Operations)
// ========================================

// GET / - Root Health / Welcome endpoint
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'Welcome to Task Manager RESTful API with MongoDB & Mongoose (Practical 5)',
    author: 'Manav Lakhani (24CS047)',
    course: 'ITUE301: Advanced Web Development Frameworks',
    database: mongoose.connection.readyState === 1 ? 'Connected (MongoDB)' : 'Connecting...',
    endpoints: {
      getAllTasks: 'GET /tasks',
      getTaskById: 'GET /tasks/:id',
      createTask: 'POST /tasks',
      updateTask: 'PUT /tasks/:id',
      deleteTask: 'DELETE /tasks/:id'
    }
  });
});

// GET /tasks - Read all tasks (supports query filtering by ?completed=, ?priority=, ?search=)
app.get('/tasks', async (req, res, next) => {
  try {
    const { completed, priority, search } = req.query;
    const filter = {};

    if (completed !== undefined) {
      filter.completed = completed === 'true';
    }
```

```
    if (priority) {
      filter.priority = priority.toLowerCase();
    }
    if (search) {
      filter.$or = [
        { title: { $regex: search, $options: 'i' } },
        { description: { $regex: search, $options: 'i' } }
      ];
    }

    const tasks = await Task.find(filter).sort({ createdAt: -1 });

    res.status(200).json({
      success: true,
      count: tasks.length,
      data: tasks
    });
  } catch (err) {
    next(err);
  }
});

// GET /tasks/:id - Read single task by ID (Supplementary Problem 3: returns 404 if not found)
app.get('/tasks/:id', validateTaskId, async (req, res, next) => {
  try {
    const task = await Task.findById(req.params.id);
    if (!task) {
      return res.status(404).json({
        error: 'Not Found',
        message: `Task with ID ${req.params.id} does not exist in the database.`
      });
    }

    res.status(200).json({
      success: true,
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// POST /tasks - Create a new task in MongoDB using Task.create() / new Task().save()
app.post('/tasks', async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;

    // Notice: Mongoose pre-save hook handles whitespace trimming on title
    // Mongoose schema enforces required: true and priority enum
    const task = new Task({
      title,
      description,
      completed,
      priority
    });
```

```
    const savedTask = await task.save();

    res.status(201).json({
      success: true,
      message: 'Task created successfully in MongoDB database.',
      data: savedTask
    });
  } catch (err) {
    next(err);
  }
});

// PUT /tasks/:id - Update an existing task using Task.findByIdAndUpdate()
app.put('/tasks/:id', validateTaskId, async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;
    const updates = {};

    if (title !== undefined) updates.title = title;
    if (description !== undefined) updates.description = description;
    if (completed !== undefined) updates.completed = completed;
    if (priority !== undefined) updates.priority = priority;

    const updatedTask = await Task.findByIdAndUpdate(
      req.params.id,
      updates,
      { new: true, runValidators: true }
    );

    if (!updatedTask) {
      return res.status(404).json({
        error: 'Not Found',
        message: `Task with ID ${req.params.id} not found.`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task updated successfully in MongoDB.',
      data: updatedTask
    });
  } catch (err) {
    next(err);
  }
});

// DELETE /tasks/:id - Delete a task using Task.findByIdAndDelete()
app.delete('/tasks/:id', validateTaskId, async (req, res, next) => {
  try {
    const deletedTask = await Task.findByIdAndDelete(req.params.id);

    if (!deletedTask) {
      return res.status(404).json({
        error: 'Not Found',
```

```
        message: `Task with ID ${req.params.id} not found.`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task deleted successfully from MongoDB.',
      data: deletedTask
    });
  } catch (err) {
    next(err);
  }
});

// ========================================
// 6. Deliberate Error Trigger (Faculty Demonstration)
// ========================================
app.get('/trigger-error', (req, res, next) => {
  const err = new Error('Demonstration: Deliberately triggered internal server error.');
  err.status = 500;
  next(err);
});

// ========================================
// 7. Custom 404 Handler for Undefined Routes
// ========================================
app.use((req, res) => {
  res.status(404).json({
    error: 'Route Not Found',
    message: `Cannot ${req.method} ${req.originalUrl}. This endpoint does not exist.`,
    validRoutes: [
      'GET    /tasks',
      'GET    /tasks/:id',
      'POST   /tasks',
      'PUT    /tasks/:id',
      'DELETE /tasks/:id',
      'GET    /trigger-error'
    ]
  });
});

// ========================================
// 8. Global Centralized Error Handling Middleware
// Formats Mongoose ValidationErrors and CastErrors cleanly
// ========================================
app.use((err, req, res, next) => {
  console.error('[Global Error Handler] Caught Exception:\n', err.stack || err.message);

  // Mongoose Validation Error (e.g. required field missing, enum mismatch)
  if (err.name === 'ValidationError') {
    const errorDetails = Object.values(err.errors).map(e => e.message);
    return res.status(400).json({
      error: 'Validation Error',
      message: errorDetails.join(', '),
      fields: Object.keys(err.errors)
```

```
    });
  }

  // Mongoose Cast Error (e.g. bad ObjectId format)
  if (err.name === 'CastError') {
    return res.status(400).json({
      error: 'Invalid Parameter',
      message: `Invalid format for field '${err.path}': ${err.value}`
    });
  }

  const statusCode = err.status || 500;
  res.status(statusCode).json({
    error: err.name || 'Internal Server Error',
    message: err.message || 'Something went wrong on the server.'
  });
});

// ========================================
// 9. Database Connection & Server Initialization
// ========================================
let serverInstance = null;

const startServer = async () => {
  try {
    await mongoose.connect(MONGO_URI);
    console.log(`[MongoDB] Connected successfully to: ${MONGO_URI}`);

    if (require.main === module) {
      serverInstance = app.listen(PORT, () => {
        console.log(`==================================================`);
        console.log(`Task Manager REST API (Practical 5) running on port ${PORT}`);
        console.log(`Base URL: http://localhost:${PORT}`);
        console.log(`MongoDB Database: Connected via Mongoose`);
        console.log(`==================================================`);
      });
    }
  } catch (err) {
    console.error('[MongoDB] Connection failed:', err.message);
  }
};

startServer();

module.exports = app;
```

3. test-api.js (Automated Verification Test Suite):

```
require('dotenv').config();
const mongoose = require('mongoose');
const app = require('./server');

let server;
const PORT = 5056;
const BASE_URL = `http://localhost:${PORT}`;
```

```
async function request(path, options = {}) {
  const url = `${BASE_URL}${path}`;
  const response = await fetch(url, options);
  let data;
  try {
    data = await response.json();
  } catch (e) {
    data = await response.text();
  }
  return {
    status: response.status,
    headers: Object.fromEntries(response.headers.entries()),
    data
  };
}

async function runTests() {
  console.log('============================================================');
  console.log('  Practical 5: MongoDB & Mongoose REST API Verification Tests ');
  console.log('  Student: Manav Lakhani (24CS047) | ITUE301                        ');
  console.log('============================================================\n');

  // Wait for MongoDB connection
  if (mongoose.connection.readyState !== 1) {
    await new Promise((resolve) => mongoose.connection.once('open', resolve));
  }

  let passed = 0;
  let total = 0;

  async function testCase(name, fn) {
    total++;
    try {
      await fn();
      console.log(`[PASS] Test ${total}: ${name}`);
      passed++;
    } catch (err) {
      console.error(`[FAIL] Test ${total}: ${name}`);
      console.error(`       Error: ${err.message}`);
    }
  }

  // 1. Health check
  await testCase('GET / - API root returns 200 OK and MongoDB status', async () => {
    const res = await request('/');
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);
    if (!res.data.endpoints) throw new Error('Endpoints object missing');
  });

  // 2. POST /tasks without Content-Type header (Content-Type middleware)
  await testCase('POST /tasks without Content-Type header rejected with 400 Bad Request', async ()
=> {
    const res = await request('/tasks', {
      method: 'POST',
```

```
      body: JSON.stringify({ title: 'No Header Task' })
    });
    if (res.status !== 400) throw new Error(`Expected 400, got ${res.status}`);
    if (res.data.error !== 'Bad Request') throw new Error(`Expected Bad Request error`);
  });

  // 3. POST /tasks with valid payload (MongoDB persistence)
  let createdTaskId;
  await testCase('POST /tasks creates task in MongoDB and returns 201 Created', async () => {
    const payload = {
      title: 'Practical 5 MongoDB Task',
      description: 'Verifying Mongoose schema and database insertion',
      completed: false,
      priority: 'high'
    };
    const res = await request('/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });
    if (res.status !== 201) throw new Error(`Expected 201, got ${res.status}`);
    if (!res.data.data._id) throw new Error('MongoDB _id is missing');
    if (res.data.data.title !== payload.title) throw new Error('Title mismatch');
    if (res.data.data.priority !== 'high') throw new Error('Priority mismatch');
    createdTaskId = res.data.data._id;
  });

  // 4. Schema Validation: Missing required title field
  await testCase('POST /tasks without required title returns 400 structured Validation Error',
async () => {
    const res = await request('/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ description: 'Missing title task' })
    });
    if (res.status !== 400) throw new Error(`Expected 400, got ${res.status}`);
    if (res.data.error !== 'Validation Error') throw new Error(`Expected Validation Error, got
${res.data.error}`);
    if (!res.data.message.includes('Task title is required')) throw new Error('Expected required
message');
  });

  // 5. Schema Validation: Invalid priority enum (Supplementary Problem 1)
  await testCase('POST /tasks with invalid priority enum returns 400 Validation Error', async ()
=> {
    const res = await request('/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        title: 'Enum Test Task',
        priority: 'urgent' // Not in ['low', 'medium', 'high']
      })
    });
    if (res.status !== 400) throw new Error(`Expected 400, got ${res.status}`);
    if (res.data.error !== 'Validation Error') throw new Error(`Expected Validation Error`);
```

```
    if (!res.data.message.includes('Priority must be one of')) throw new Error('Expected enum
error message');
  });

  // 6. Pre-save hook: Automatic title whitespace trimming (Supplementary Problem 2)
  let trimmedTaskId;
  await testCase('POST /tasks automatically trims title whitespace via pre-save hook', async () =>
{
    const untrimmedTitle = '    Whitespace Trimmed Task Title    ';
    const res = await request('/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: untrimmedTitle })
    });
    if (res.status !== 201) throw new Error(`Expected 201, got ${res.status}`);
    if (res.data.data.title !== 'Whitespace Trimmed Task Title') {
      throw new Error(`Expected trimmed title, got '${res.data.data.title}'`);
    }
    trimmedTaskId = res.data.data._id;
  });

  // 7. GET /tasks - Read all tasks
  await testCase('GET /tasks retrieves tasks from MongoDB and returns 200 OK', async () => {
    const res = await request('/tasks');
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);
    if (!Array.isArray(res.data.data)) throw new Error('Expected array of tasks');
    if (res.data.count < 1) throw new Error('Expected at least 1 task in database');
  });

  // 8. GET /tasks/:id - Read single task by ID
  await testCase('GET /tasks/:id with valid existing ID returns 200 OK', async () => {
    const res = await request(`/tasks/${createdTaskId}`);
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);
    if (res.data.data._id !== createdTaskId) throw new Error('ID mismatch');
  });

  // 9. GET /tasks/:id - Non-existent ID returns 404 (Supplementary Problem 3)
  await testCase('GET /tasks/:id with non-existent ObjectId returns 404 Not Found', async () => {
    const fakeId = '507f1f77bcf86cd799439011';
    const res = await request(`/tasks/${fakeId}`);
    if (res.status !== 404) throw new Error(`Expected 404, got ${res.status}`);
    if (res.data.error !== 'Not Found') throw new Error('Expected Not Found error');
  });

  // 10. GET /tasks/:id with malformed ObjectId
  await testCase('GET /tasks/:id with malformed ObjectId rejected with 400 Bad Request', async ()
=> {
    const res = await request('/tasks/invalid-id-123');
    if (res.status !== 400) throw new Error(`Expected 400, got ${res.status}`);
    if (res.data.error !== 'Invalid Task ID') throw new Error('Expected Invalid Task ID');
  });

  // 11. PUT /tasks/:id - Update task in MongoDB
  await testCase('PUT /tasks/:id updates task in MongoDB and returns 200 OK', async () => {
    const res = await request(`/tasks/${createdTaskId}`, {
```

```
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ completed: true, priority: 'low' })
    });
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);
    if (res.data.data.completed !== true) throw new Error('Completed not updated');
    if (res.data.data.priority !== 'low') throw new Error('Priority not updated');
  });

  // 12. DELETE /tasks/:id - Remove task from MongoDB
  await testCase('DELETE /tasks/:id removes task from MongoDB and returns 200 OK', async () => {
    const res = await request(`/tasks/${createdTaskId}`, {
      method: 'DELETE'
    });
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);

    // Verify task is deleted from database
    const verifyRes = await request(`/tasks/${createdTaskId}`);
    if (verifyRes.status !== 404) throw new Error('Deleted task still found in database');
  });

  // Clean up the trimmed test task
  if (trimmedTaskId) {
    await request(`/tasks/${trimmedTaskId}`, { method: 'DELETE' });
  }

  // 13. Undefined route custom 404 handler
  await testCase('GET /undefined-route returns structured 404 JSON response', async () => {
    const res = await request('/api/undefined-route');
    if (res.status !== 404) throw new Error(`Expected 404, got ${res.status}`);
    if (res.data.error !== 'Route Not Found') throw new Error('Expected Route Not Found');
  });

  // 14. Centralized global error handler
  await testCase('GET /trigger-error triggers centralized 500 error handler', async () => {
    const res = await request('/trigger-error');
    if (res.status !== 500) throw new Error(`Expected 500, got ${res.status}`);
  });

  console.log('\n============================================================');
  console.log(`  Test Results Summary: ${passed} / ${total} Tests Passed (${Math.round((passed /
total) * 100)}%)`);
  console.log('============================================================\n');

  server.close();
  await mongoose.disconnect();
}

server = app.listen(PORT, () => {
  runTests();
});
```

4. .env.example & package.json:

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/task_manager_db
```

GitHub Repository Link: https://github.com/24CS047Manav/Awdf_Practica_5

Output

1. MongoDB Connection & ODM Integration (Step 2):
Successfully connected Express to the local MongoDB database engine at mongodb://127.0.0.1:27017/task_manager_db using Mongoose ODM and dotenv for environment variable abstraction.

2. Mongoose Schema Definition & Model Creation (Step 3):
Defined Task schema in models/Task.js enforcing four core fields: title (String, required), description (String), completed (Boolean, default: false), and createdAt (Date, default: Date.now).

3. Supplementary Problem 1 - Priority Field Enum Validation:
Added priority field restricted strictly to an enum of ['low', 'medium', 'high'] with default 'medium'. Requests attempting invalid values (e.g. 'urgent') are rejected with structured 400 Validation Error.

4. Supplementary Problem 2 - Pre-Save Title Trimming Hook:
Implemented a Mongoose pre-save middleware hook (taskSchema.pre('save')) that automatically sanitizes and trims leading/trailing whitespace from the title before persisting into MongoDB.

5. Create Document (POST /tasks - 201 Created):
Creates and saves document in MongoDB collection tasks with auto-generated _id and default timestamps.

6. Read Documents (GET /tasks - 200 OK):
Queries all documents from MongoDB using Task.find() with sorting and optional filtering by status and priority.

7. Read Single Document (GET /tasks/:id - 200 OK / 404 Not Found):
Supplementary Problem 3: Validates 24-character hex ObjectId and returns 200 OK if found, or clean 404 Not Found JSON if the document does not exist.

8. Update Document (PUT /tasks/:id - 200 OK):
Updates existing document in MongoDB using Task.findByIdAndUpdate() with runValidators: true to ensure validation rules apply to updates.

9. Delete Document (DELETE /tasks/:id - 200 OK):
Removes document from MongoDB using Task.findByIdAndDelete() and returns the deleted entity.

10. Structured Mongoose Validation Error Responses (Step 5):
Centralized error handler intercepts Mongoose ValidationError and CastError objects, returning clean, readable JSON error summaries instead of unformatted raw stack traces.

11. Automated Verification & Test Results (100% Pass Rate):
Executed test suite covering all 14 test scenarios with 100% pass rate validating real MongoDB persistence, pre-save hooks, enum constraints, and status codes.

Test Suite Execution Output Log:

```
============================================================
  Practical 5: MongoDB & Mongoose REST API Verification Tests
```

```
  Student: Manav Lakhani (24CS047) | ITUE301
============================================================

[MongoDB] Connected successfully to: mongodb://127.0.0.1:27017/task_manager_db
[PASS] Test 1: GET / - API root returns 200 OK and MongoDB status
[PASS] Test 2: POST /tasks without Content-Type header rejected with 400 Bad Request
[PASS] Test 3: POST /tasks creates task in MongoDB and returns 201 Created
[PASS] Test 4: POST /tasks without required title returns 400 structured Validation Error
[PASS] Test 5: POST /tasks with invalid priority enum returns 400 Validation Error
[PASS] Test 6: POST /tasks automatically trims title whitespace via pre-save hook
[PASS] Test 7: GET /tasks retrieves tasks from MongoDB and returns 200 OK
[PASS] Test 8: GET /tasks/:id with valid existing ID returns 200 OK
[PASS] Test 9: GET /tasks/:id with non-existent ObjectId returns 404 Not Found
[PASS] Test 10: GET /tasks/:id with malformed ObjectId rejected with 400 Bad Request
[PASS] Test 11: PUT /tasks/:id updates task in MongoDB and returns 200 OK
[PASS] Test 12: DELETE /tasks/:id removes task from MongoDB and returns 200 OK
[PASS] Test 13: GET /undefined-route returns structured 404 JSON response
[PASS] Test 14: GET /trigger-error triggers centralized 500 error handler

============================================================
  Test Results Summary: 14 / 14 Tests Passed (100%)
============================================================
```